

G22・G55・G70の構造から CIMの解をどう理解するか

何を調べたいのか → なぜその指標か → 何が分かるか

出発点は「CIMが作る解の違いを説明したい」

CIMが同じように動いても、入力G22・G55・G70のどれかによって、得られる解の傾向は変わる。その違いを考えるため、まずグラフのどの部分を分けて考えられ、どこを詳しく調べる必要があるかを整理する。

将来はGAやSAへ渡す候補の評価に使いたい。しかし、グラフだけから計算する値は、同じグラフ上の全候補で共通である。今回は、候補を採点する前の土台として、構造とCIM出力を対応させて読む方法に集中する。

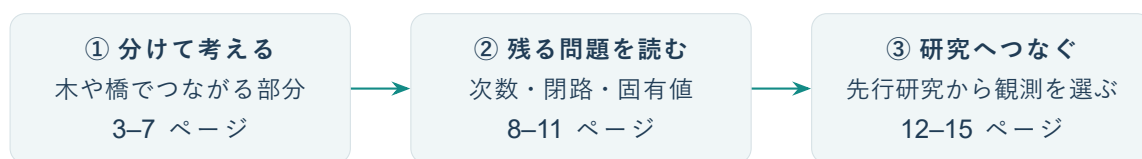


図1 この資料の道順。用語を順に覚えるためではなく、「CIMの出力のどこを見るか」を決めるために読む。2ページで共通の得点を確認する。

なぜ「橋」や「core」を説明するのか

木や橋の部分では、他の部分の得点を落とさずに、すべての辺をcutできる。そのため、CIMの未cut辺を木・橋の側と残った中心部分に分けると、改善の余地がどこにあるかを区別できる。橋やcoreは、その分け方を説明するために登場する。

最初に区別しておくこと

「構造上、分けて解ける」ことと、「現在のCIMやGAが容易に良い解を出す」ことは別である。前者は数学と構造の話、後者は実際の探索結果の話として扱う。

対象入力：本リポジトリのG22・G55・G70。手書き修正「Print (8).pdf」を反映。

今回は資料の再構成と構造の追加集計。CIM・GA・SAの新たな性能実験は行っていない。

01 / 共通の得点を確認する

異なる組を結ぶ辺が、Max-Cutの得点

この得点を、後で「部分ごと」に分けて考える。

ここでいう三つの入力は **G22・G55・G70** である。ローカルの入力ファイルでは、いずれも全辺の重みが +1 だった。したがって今回は、異なる組を結ぶ辺 1 本を 1 点と数える。

配置A：切れている辺は2本 配置B：切れている辺は4本



点の色 = 二つの組 / オレンジの辺 = cut

図 2 同じ説明用グラフの二つの割当て。緑と紺は二つの組、オレンジは cut 辺、灰色は未 cut 辺。左は 2 点、右は 4 点。

式の記号を、一つずつ読む

V 頂点の集合。各頂点に +1 か -1 を割り当てる。

E 実際に存在する辺の集合。 $\{i, j\} \in E$ は「頂点 i, j を結ぶ辺がある」の意味。全頂点对の集合ではない。

s_i 頂点 i の組を表す符号。+1 と -1 が二つの組に対応する。

w_{ij} 辺 $\{i, j\}$ の重み。今回の三つの入力では 1。

$$C_G(s) = \sum_{\{i,j\} \in E} w_{ij} \frac{1 - s_i s_j}{2}$$

両端の組	$s_i s_j$	$(1 - s_i s_j)/2$	今回の得点
同じ組	+1	0	0 点
異なる組	-1	1	1 点

符号の積と得点は別である。異なる組で -1 になるのは積 $s_i s_j$ で、得点は式に代入して 1 となる。各辺は一度だけ足す。

全反転で「変わらない」のは、どの辺か

全頂点を反転すれば、全辺の得点が変わらない。一部分だけを全反転した場合、その内部の辺の得点是不変変わらないが、外へ出る辺の得点は変わり得る。この違いが、橋の説明の鍵になる。

出典・根拠：入力の重みと構造は [7]。図と符号の説明は本資料で作成。

02 / 分けられる場所を探す

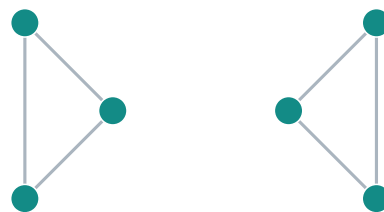
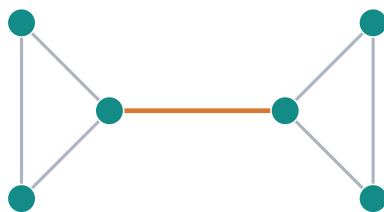
橋と関節点は、問題の「切れ目」を示す

ここでの目的は、別々に解いてから組み合わせられる場所を探すこと。

大きなグラフを小さな問題へ分けられれば、どの部分で得点を失ったかを個別に調べられる。最初に、**1本の辺または1個の頂点を境に分かれる形を見る**。

橋：オレンジの「辺」を除く

連結成分が1個から2個へ



関節点：オレンジの「点」を除く

その点につながる辺も消える

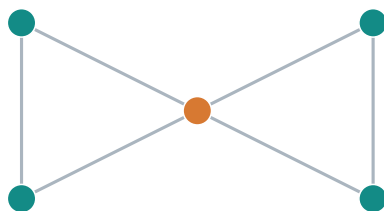


図3 説明用のグラフ。緑は頂点、灰色は辺、オレンジは除去する対象。上段は辺の除去、下段は頂点と接続辺の除去。ここで点の色は解の割当てではない。

橋：その辺が唯一の通り道

上段の中央の辺を除くと、左と右を行き来できなくなる。このような辺が橋である。三角形の辺を1本除いても、他の2本を通して戻れるため、それらは橋ではない。

関節点：その頂点が唯一の通り道

下段では、中央の頂点とそこにつながる辺を除くと左右が分かれる。このような頂点が関節点である。中央につながる各辺には三角形内の迂回路があるため、この例に橋はない。

何に使えるか

Max-Cut では、橋の左右をそれぞれ良く解いた後、片側の符号を全反転してつなげられる。関節点で分けた場合も、共有点の符号を合わせれば、各部分の最適な得点を保って結合できる。[2]

次は、最も単純な「橋が1本」の場合を数値で確かめる。

03 / 数値例から式へ

左右で4点ずつ取れたら、橋を足して9点

「OPT」という記号は、その部分だけで達成できる最大得点を表す。



図4 左右に四角形が一つずつあり、中央の橋で結ばれる。緑と紺は組、オレンジは cut 辺。右の四角形だけを全反転すると、内部8点を保ったまま橋が1点になる。

何の最大得点か	この例での値
左の四角形だけ：OPT(L)	4
右の四角形だけ：OPT(R)	4
橋1本を cut したときの得点：w	1

$$\underbrace{\text{OPT}(G)}_{\text{全体の最大得点}} = \underbrace{\text{OPT}(L)}_{\text{左内部の最大得点}} + \underbrace{\text{OPT}(R)}_{\text{右内部の最大得点}} + \underbrace{w}_{\text{橋の重み}} = 4 + 4 + 1 = 9.$$

L, R は橋を除いた後の左右の部分グラフであり、橋自身を含まない。 w はその橋の正の重みで、G22・G55・G70 では1である。

なぜ足し算の「上限」に、本当に届くのか

1. 左右を別々に最適化する。内部の得点はそれぞれの最大値になる。
2. 橋の両端が同じ組なら、右側の全頂点を反転する。
3. 右内部では $(-s_i)(-s_j) = s_i s_j$ なので得点が保たれる。橋だけは片端の符号が変わり、cut される。

これで左右と橋の最大得点を同時に達成する。左内部・右内部・橋のどれも、これ以上の得点は取れないので、合計も全体の最大値になる。

橋が複数あってもできる

橋をすべて除いてできた塊を1点に縮めると、その間の橋は輪を作らない。輪があれば迂回でき、元から橋ではないためである。端の塊から向きを合わせていけば、塊内部の得点を保って、全ての正重みの橋を cut できる。

出典・根拠：分解の一般的な利用は [2]。4+4+1 の例と説明は本資料。

04 / G55・G70 に当てはめる

中心部分に付く木と、離れている木を分ける

「橋がたくさんある」を、具体的な姿と辺数で読む。

今回の入力を調べると、G55・G70 はそれぞれ一つの中心部分と木の部分、孤立点に分かれる。中心部分を H_{55} , H_{70} と呼ぶ。旧版の「非自明ブロック」は、ここではこの中心部分を指す。

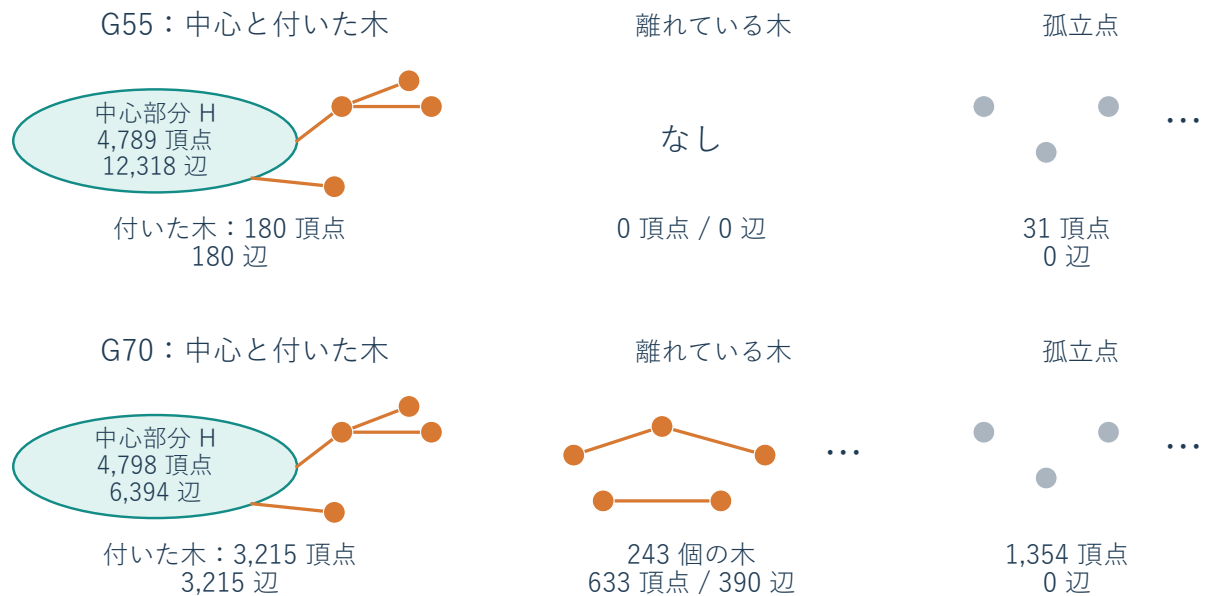


図5 入力から確かめた内訳。緑の楕円は中心部分をまとめた表示、橙の点・辺は木、灰の点は孤立点。枝の形や本数は説明用に省略しており、実測値は数字で示す。橙色の辺は全て橋。

G55：中心部分の外の 180 本を、別に数えられる

付いている木が 180 辺あり、離れた木はない。孤立点 31 個の得点は 0。

$$\text{OPT}(G55) = \text{OPT}(H_{55}) + 180.$$

G70：3,605 本が、二つの場所に分かれる

付いている木が 3,215 辺、離れている 243 個の木が計 390 辺ある。

$$\text{OPT}(G70) = \text{OPT}(H_{70}) + \underbrace{3215 + 390}_{3605\text{本の橋}}.$$

橋の本数と、木の頂点数は別の数である。例えば、離れた木は計 633 頂点だが、辺は $633 - 243 = 390$ 本である。

CIM の解を見るとき使い道

未 cut の橋があれば、その部分には改善できる余地がある。中心部分の未 cut 辺とは分けて数えたい。ただし、現在の CIM がこの反転処理を自動で行う、という意味ではない。

出典・根拠：同梱 decomposition.json。入力から今回再集計。中心部分の最適値を新たに解いたわけではない。

05 / 中心部分を取り出す手順

まず木の先端を外す。その残りが 2-core

core を導入する目的は、「どこに注目して解を調べるか」を絞ること。

前ページのように木を分けるには、今つながっている辺が **0** 本または **1** 本の点を取り除き、同じ操作を繰り返せばよい。そこで残った部分を **2-core** と呼ぶ。「2」は、残った内部で各頂点が少なくとも 2 本の辺を持つことを表す。

① 最初：次数 0・1 を除く

② A も次数 1 になった

③ どの点も次数 2 で止まる

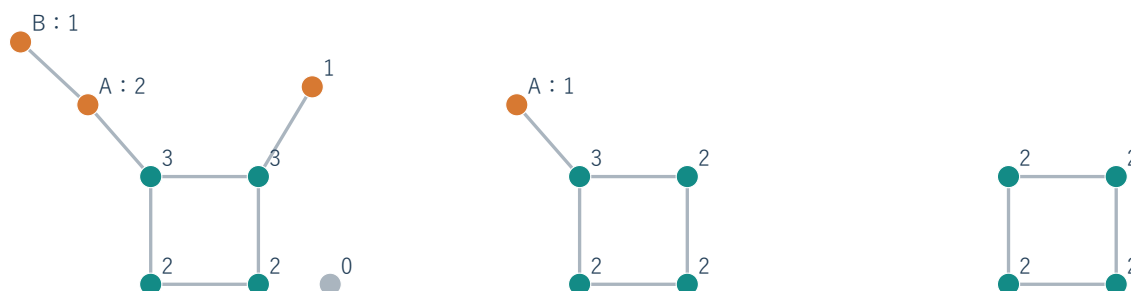


図 6 説明用の除去手順。数字はその時点で残る辺の本数。緑は最終的に残る点、橙は途中で外れる点、灰は元から孤立した点。色は解の割当てではない。

点 A を追うと、「繰り返す」理由が分かる

1. 最初の A には辺が 2 本あるため、まだ外れない。先端の B は 1 本なので外れる。
2. B につながる辺がなくなり、A の辺は 1 本になる。次は A も外れる。
3. 四角形の点は、互いにつながる 2 本ずつが残るため、これ以上外れない。

「元から次数が 2 以上の点だけを一度選ぶ」のではない。隣が消えるたびに辺数を数え直す。これが、木の長い枝も先端から取り除ける理由である。

Max-Cut で、何がうれしいのか

外した木は、中心部分の割当てが決まった後、接続点と逆の符号を順に付ければ全辺を cut できる。CIM 出力の分析でも、木の側の失点と **2-core** 内の失点を分けられる。今回は G55・G70 の 2-core が、前ページの中心部分と一致した。

残ること自体は、難しさを意味しない

図の四角形は 2-core に残るが、交互に二色を割り当てれば 4 辺とも cut できる。**2-core** は調べる範囲を絞る道具であり、難しさの点数ではない。残った中で、全辺を同時に cut できるかを妨げる構造は 9 ページで説明する。

出典・根拠：core の定義は [8]。一般の 2-core には橋が残る場合もある。今回の「中心部分と一致」は入力を確認した結果。

06 / 次数と core を区別する

同じ4本でも、周囲が残るかどうかは違う

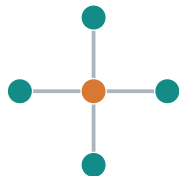
「周囲も一緒に残る」を、除去の途中経過で確かめる。

次数は、その頂点に接続する辺の本数である。一方、**core** は、辺数の少ない点を繰り返し外した後の部分を指す。同じ次数でも、隣の点のつながり方によって結果が変わる。

A：最初は 4 本

葉を除くと 0 本

中心も除かれて空



残る点なし

B：最初は 4 本

葉を除いても 3 本ずつ

互いの辺が残ри、停止

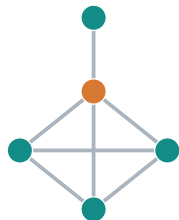


図7 両方とも橙の注目点の次数は最初 4。2-core を求める操作を左から右へ進めた。上は隣の 4 点が葉なので全て消え、中心も消える。下は葉 1 点を除いても、残る 4 点が互いに 3 本ずつの辺で支え合う。

「k-core」と「core number」の意味

k を決め、残っている内部で辺が k 本未満の点を繰り返し外す。最後に残る部分が k -core である。その点が残れる最大の k を **core number** と呼ぶ。

図の上の注目点は、1 本未満を除く条件では残るが、2 本未満を除く条件では消えるため、core number は 1。下の注目点は 3 本未満を除いても残るが、4 本未満を除くと消えるため、core number は 3 である。

14-core : 1,733 頂点

G22 の 2-core : 2,000 頂点

図8 G22 の包含関係。面積は比率を表さない。14-core の点は 2-core にも含まれる。

14-core に入るとは、その部分の中で各点が少なくとも 14 本の辺を保てること。「元の次数がちょうど 14」の意味ではない。したがって「G22 の全頂点が 2-core に残る」と「多くが 14-core にも残る」は両立する。

出典・根拠：[7][8]。次は、実際の辺数を表す次数で三つの入力比べる。

07 / 実データの比較

G22 は辺が多く、G70 は木の部分が多い

構造の違いと、実験で良く解けたかを、同じ意味にしない。

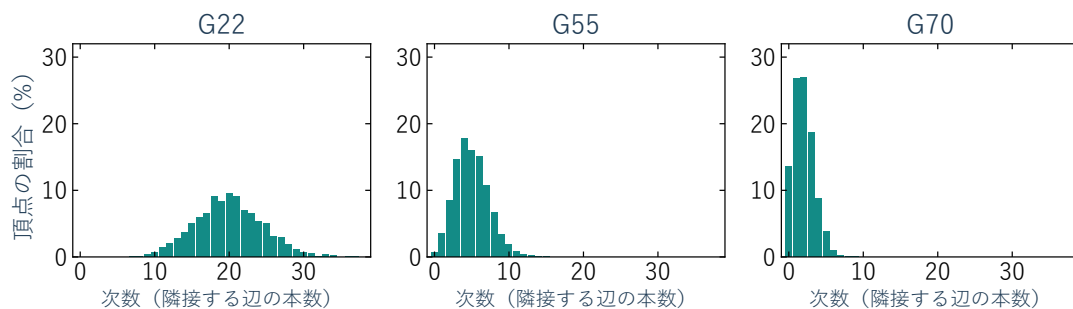


図 9 実入力の次数分布。横軸は隣接する辺の本数、縦軸は全頂点に占める割合。G22 は平均約 20 本、G55 は約 5 本、G70 は約 2 本。core number の分布ではない。

比較する量	G22	G55	G70
全頂点数	2,000	5,000	10,000
全辺数	19,990	12,498	9,999
2-core の頂点数	2,000	4,789	4,798
2-core 内の辺数	19,990	12,318	6,394
橋の本数	0	180	3,605

この比較から分かることは、二つ

G22 には橋がなく、この切り分けでは全体が残る。G70 では 3,605 辺を橋として別に扱える。また、G55 と G70 の 2-core は頂点数がほぼ同じでも、内部の辺数は大きく違う。中心部分の頂点数だけで、同じ問題とは言えない。

では、なぜ G22 のほうが良く解けたのか

既存 GA 実験では、次のように G22 の最終得点が基準値に近かった。ただし、理由はまだ特定できていない。

基準値 BKS との差 (%)	G22	G55	G70
ランダム初期化	0.005	0.687	1.377
全 CIM 初期化	0.007	0.845	0.999

各条件 16 run・300 世代後の平均 best-cut を用い、 $100 \times (\text{BKS} - \text{cut}) / \text{BKS}$ で表示。BKS は記録された最良既知値で、厳密な最適値とは限らない。[7]

ここで答えられること

「2-core が大きいほど、この GA で解きにくい」という単純な説明は、この結果に合わない。分けやすさだけでは探索結果を説明できないので、次に中心部分の輪の構造と、CIM の振幅の偏りを見る。

出典・根拠：構造・次数は [7]。GA は 2026 年 9 月 14 日の既存記録。CIM 単独の難易度測定ではない。

08 / 中心部分の中を見る

奇数の輪では、最後の1辺が両立しない

2-coreに残った後は、辺どうしの要求が矛盾する場所に注目する。

全辺を cut するには、隣り合う点を違う組に入れる必要がある。四角形では交互に割り当てて元へ戻れるが、三角形や五角形では、最後の辺の両端が同じ組になる。

四角形：4 / 4 本

三角形：最大2 / 3 本

五角形：最大4 / 5 本

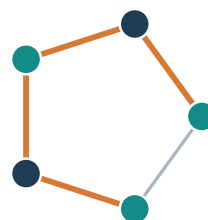
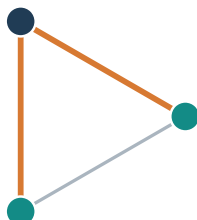
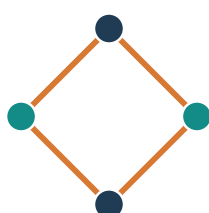
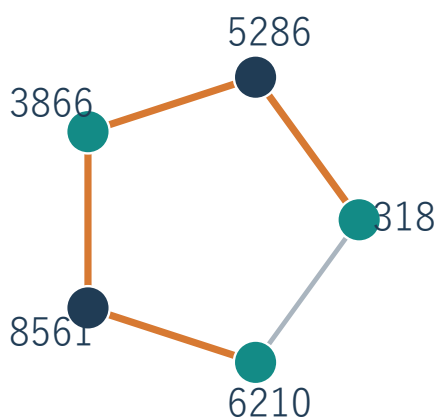


図 10 説明用の輪。緑・紺は組、橙は cut 辺、灰は未 cut 辺。四角形は 4 本全てを cut できるが、三角形は最大 2 本、五角形は最大 4 本。

2-core だけでは分からなかった違い

四角形も三角形も、各点の次数は 2 で、どちらも 2-core に残る。しかし、全辺を同時に cut できるかは異なる。「残る点の数」だけでなく、「どんな輪があるか」まで見る必要がある。



G70 にも、実際に 5 頂点の輪がある。

318 → 5286 → 3866 → 8561 → 6210 → 318。入力に 5 辺とも存在することを確認した。G70 には三角形がなくても、全辺を同時に cut できるわけではない。

図 11 G70 で確認した閉路。数字は入力の頂点番号。位置と割当ては説明用で、CIM 出力を描いたものではない。

奇数の輪を数えるだけでは、失点は決まらない

二つの奇数閉路が同じ辺を共有すると、その 1 辺を未 cut にすることで両方の矛盾を解消できることがある。したがって「奇数閉路が 10 個あるから 10 点失う」とは数えられない。閉路どうしの重なり方と、実際の未 cut 辺を対応させて見る。

ここまでは、辺と符号の組合せを見た。次は、CIM の連続的な振幅がどんな形に増えやすいかを見る。

出典・根拠：図 10 は説明用。図 11 は実入力の確認結果 [7]。

09 / CIM の振幅へ話を進める

固有ベクトルは「同じ倍率で変わる配置」

CIM では結合を通じて振幅が変わる。その変化の方向を読む準備。

隣の値を足す操作を考える。グラフの辺がある場所を 1、ない場所を 0 とした表が隣接行列 A である。頂点上の値を並べた x に A を掛けると、各頂点に隣の値の合計が入る。



図 12 四角形に交互の数値を置く例。緑・紺は正負、灰は接続。どの点でも、隣を足した結果は元の値の -2 倍になる。

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix}, \quad Ax = \begin{pmatrix} -2 \\ 2 \\ -2 \\ 2 \end{pmatrix} = -2x.$$

行列では左上から時計回りに頂点を並べた。各点がばらばらに変わるのでなく、配置全体が同じ倍率で変わっている。このとき、配置 x が固有ベクトル、倍率 -2 が固有値である。

Max-Cut との接点は、隣と逆符号になる方向

正重みの辺を cut するには、隣と異なる符号が望ましい。隣接行列 A の負側の固有値に対応する配置は、そのような関係を読む手掛かりになる。[3] ただし一般には値の大きさも点ごとに違い、符号だけを取れば必ず最適解になるわけではない。

CIM では、何に使うのか

結合によって増えやすい振幅の配置と、CIM が実際に作った配置を比べたい。固有値はそのための道具の一つである。どの行列を使った固有値かを、次ページで区別する。

出典・根拠：スペクトルと Max-Cut の接点は [3]。4 点の計算は本資料の説明例。

10 / 行列の役割を分ける

つながり方と、CIM の増幅は別の問い

「固有値が大きい・小さい」の前に、何を測るかを定める。

問い A：二つの塊は、どれほどつながっているか

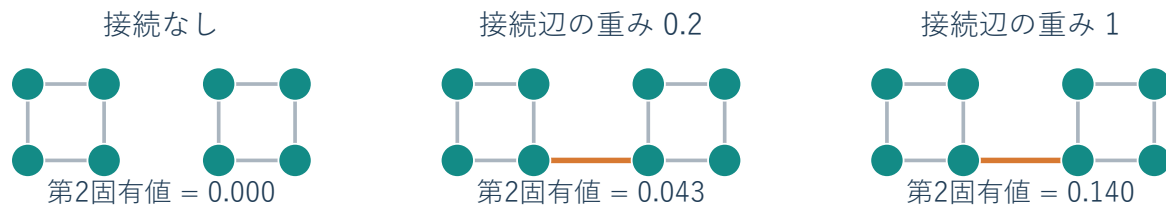


図 13 同じ二つの四角形を、重み 0、0.2、1 の辺でつなぐ説明例。緑は頂点、灰は重み 1 の辺、橙は接続辺。値は正規化ラプラシアン第 2 小固有値。G-set での測定値ではない。

この例では接続が強いほど値が増える。正規化ラプラシアンは $L = I - D^{-1/2}AD^{-1/2}$ で、 D は各点につながる重みの合計を対角に並べた行列である。辺を持つ連結グラフでは、第 2 小固有値は正になる。

G55・G70 には離れた成分がある。その全体で 0 が出ても、中心部分のつながり方を比較したことにはならない。比較対象を中心部分などにそろえてから使う。

問い B：CIM では、どの振幅の配置が増えるか

本リポジトリの Max-Cut 結合は $J = -\alpha A$ ($\alpha > 0$)。正の隣接行列 A とは符号が逆である。[7]

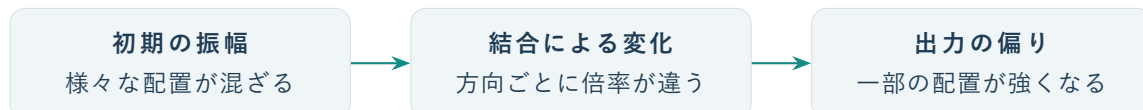


図 14 増幅の考え方を示す説明図。全ての CIM 出力がこの一つの要因だけで決まる、という意味ではない。

初期の小振幅を線形近似し、一回の更新を $c_{t+1} \simeq g_t(aI + bJ)c_t + \text{雑音}$ と書く。 c_t は全頂点の振幅、 a は自分の振幅を残す係数、 b は結合の係数、 g_t は共通の増幅を表す。 $Jv = \mu v$ の方向は、同じ近似で $g_t(a + b\mu)$ 倍される。これが固有方向を調べる理由である。

どこまで分かっていて、どこからが仮説か

既存解析では G55・G70 の特定の固有方向への偏りが観測された。[7] ただし実際の CIM には飽和・雑音・時間変化がある。固有値だけで G22 との成績差を説明し切ったわけではない。

出典・根拠：[7]。線形近似は増幅の説明用で、実装の完全な更新式ではない。

11 / 最初に読む先行研究

MQLib：構造から、合う探索法を選ぶ

ここまで見た構造量が、探索法との相性を読む助けになるかを考える。

何をした研究か

Dunning・Gupta・Silberholz は、Max-Cut/QUBO の 37 探索法を比較し、58 種類の問題特徴から、どの探索法が良い結果を出そうかを予測した。[1] それぞれの手法に対して、5 回の試行平均が比較対象中で最良になるかを学ぶランダムフォレストを作る。

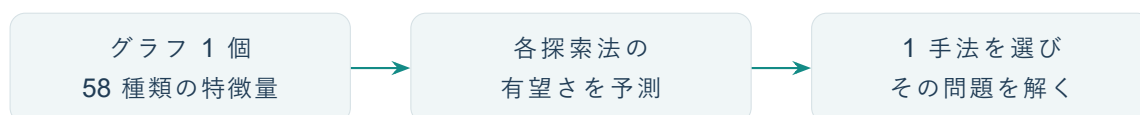


図 15 論文の処理を整理した図。選ぶ対象は探索法。同じグラフ上の個々の CIM 候補を選ぶ実験ではない。

何が分かったか

2,635 問題を学習 1,844・評価 791 に分けた実験で、1 手法を選ぶ構成の平均期待偏差は 0.08%。比較に使われた BUR02 という 1 手法を全問題で固定使用した場合は 0.27% だった。[1, Section 6]

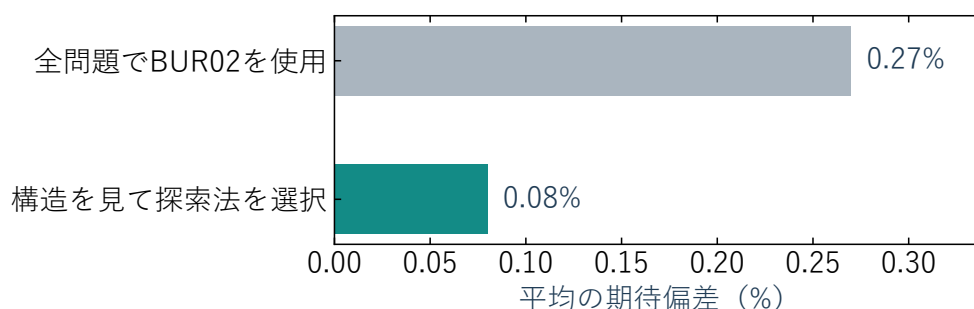


図 16 論文の報告値を再描画。小さいほど論文内の最良既知値に近い。灰は BUR02 固定、緑は問題ごとに探索法を選んだ場合。本研究での再実験ではない。

この研究から借りたい考え方

「普遍的な難しさ」を 1 個の数字にする前に、ある構造で、ある探索法がどう振る舞うかを見る。今回なら、グラフ構造と CIM 条件を対応させ、どこに未 cut 辺や配置の偏りが残るかを調べる。

読む場所と、まだ言えないこと

Section 2.2 で特徴量、Section 6 で探索法選択と評価を読む。特徴量で手法の相性を予測できたことは、個々の CIM 解の「将来良い種になる価値」まで証明したことではない。

出典・根拠：[1] の著者公開本文。偏差の基準は最良既知値であり、全問題の厳密最適値ではない。

12 / 解の状態も見る先行研究

ECO-DQN：今の配置から、次の一手を選ぶ

グラフ構造に加え、現在の解の情報をどう使うかを見る。

ECO-DQN は、1 頂点を選んで符号を反転する操作を繰り返す。前に動かした頂点を再び動かせるので、一度の判断をやり直せる。MPNN は辺を通じて情報を集め、各頂点の反転に対する Q 値を予測する。[6]

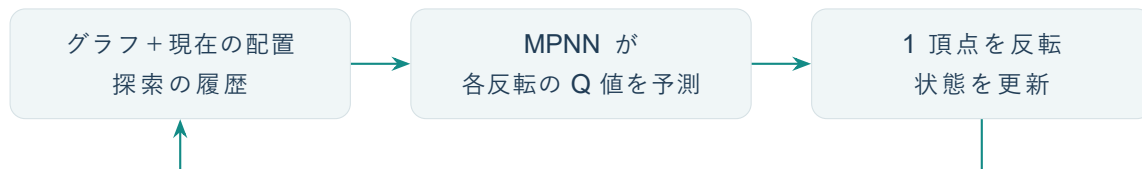


図 17 ECO-DQN の 1 ステップ。動くたびに状態が変わり、評価をやり直す。[6]

「すぐ増える得点」と「Q 値」は違う

ある頂点に未 cut 辺が 3 本、cut 辺が 1 本つながっていれば、その頂点の反転で 3 本が cut になり、1 本が未 cut になる。直後の変化は $3 - 1 = +2$ 点。この即時の改善量 (gain) は、学習せずに計算できる。

一方、Q 値はその一手の後にも探索が続けたときの報酬を予測する。ECO-DQN では過去最高 cut を更新した分を頂点数で割って主報酬とし、初めて訪れる局所最良状態への補助報酬も使う。一時的な悪化には直接罰を与えず、先の改善を探れるようにしている。[6]

モデルは、何を見ているか

頂点ごと：現在の符号、反転直後の gain、最後の反転からの手数。

全体：過去最良 cut との差、過去最良配置との距離、すぐ改善できる反転の数、残り手数。

この 7 観測から、構造だけでなく今どこまで探索が進んだかも読む。

今回の関心に近いのは、どの部分か

CIM 出力についても「どこに改善できる点が残るか」は計算できる。まずは ECO-DQN の観測の考え方を借り、CIM の解を理解する診断に使う。GA/SA に渡した後の価値を Q 値と同一視はしない。

出典・根拠：[6] の Observations・Rewards。3 対 1 の例は本資料の説明例。

13 / ECO-DQN の結果を正しく読む

参考になる成果と、今回まだ確かめていないこと

CIM との関係を、実験済みの事実と応用案に分ける。

論文で報告されたこと

200 頂点の ER グラフで学習したモデルを G-set に適用した Table 1 では、「G22-32」群の平均比率が ECO-DQN で 0.971、S2V-DQN で 0.919 だった。[6] 基準 cut 値に対する得点の比であり、**G22** 単独の結果でも、最適解に到達する確率でもない。この群は各グラフ 1 エピソードの評価である。

補足資料には **SimCIM** との比較もある。ただし、CIM が作った解を ECO-DQN に渡す結合実験ではなく、本リポジトリの進行波 CIM と SimCIM も同一実装ではない。



図 18 今回の資料からの利用案。CIM を解生成の中心に置き、観測量で出力を診断する。ここに ECO-DQN 学習器を実装済みという意味はない。

そのまま導入する前に残る、三つの課題

1. **Q 値の意味が違う**。ECO-DQN 自身の報酬と行動方針に対する予測であり、GA/SA の初期解としての将来価値ではない。
2. **CIM 出力には探索履歴がない**。「最後の反転から何手」などの観測は、そのままでは決まらず、初期化を設計する必要がある。
3. **対象と計算費用が違う**。学習時のグラフ分布から G55・G70 へ移す性能や、10,000 頂点で毎手推論する負荷は、この資料では未検証である。

現段階で取り入れる範囲

まずは学習器を追加せず、反転 gain や未 cut 辺の位置を CIM 出力から計算する。その結果で不足する情報が分かってから、学習モデルが必要かを判断する。

出典・根拠：報告値・SimCIM 比較は [6]。CIM 診断への利用は本資料の提案。

14 / 話を研究目的へ戻す

次に知りたいのは、CIMが「どこで失点するか」

構造の説明を、測れる問いへつなぐ。

ここまでで、G55・G70を木と中心部分に分ける意味、次数とcoreの違い、中心部分の閉路、振幅の固有方向を整理した。これらを使い、まず三つの問いを立てる。

問い1：失点は、木と中心部分のどちらにあるか

各CIM解の未cut辺を分けて数える。橋側の未cutは、部分全反転で改善できる余地を示す。中心部分の未cutは、閉路の制約と実際の探索状態を調べる対象になる。

問い2：異なる解が、同じ場所で失敗していないか

全頂点の符号を単純に比較するだけでなく、どの辺がcut・未cutになったかを比べる。孤立点の符号や全体反転による違いを除いて、中心部分の共通した配置を見る。

問い3：配置の偏りは、振幅の増幅と対応するか

CIMの結合行列 J の固有方向と、保存された振幅や符号配置を照合する。特定の方向へ寄ることが、得点や未cut辺の位置とどう関係するかを調べる。

Transformerは、何を足せそうかが見えてから

GraphGPS [4]は局所の情報交換と全体のattentionを組み合わせる。MARCO [5]はGraph Transformerを組合せ最適化の探索に使う。ただし、モデル名を先に決めるより、次数・gain・閉路などの観測だけでは表せない情報が何かを先に明らかにしたい。

将来の候補評価へ戻る条件

構造とCIM出力の関係が整理できた後で、「その情報がGA/SAへ渡す初期解の良さを予測するか」を検証する。同じグラフでは構造量だけで候補の差は付かないため、候補ごとの配置情報との組合せが必要になる。価値関数や学習データの詳細設計は、その段階の課題として残す。

この資料の結論の範囲

G22の成績が良い原因や、CIM解の将来価値を予測する評価関数が完成したわけではない。今回できたのは、どの構造の、どの振る舞いを観測すれば説明を進められるかを具体化したことである。

15 / 参照先と再現情報

本文で使った根拠を、たどれるようにする

報告値・入力を集計・説明用の例を区別する。

[1] Dunning, I.; Gupta, S.; Silberholz, J. (2018).

What Works Best When? A Systematic Evaluation of Heuristics for Max-Cut and QUBO. INFORMS Journal on Computing, 30(3), 608–624.

[著者公開本文 / 公式実装](#)。Section 2.2・6。図 16 は報告値の再描画。

[2] Rehfeldt, D.; Koch, T.; Shinano, Y. (2023).

Faster exact solution of sparse MaxCut and QUBO problems. Mathematical Programming Computation, 15, 445–470.

[公開本文・DOI](#)。Section 4.1 の分解を参照。橋の数値例は本資料で作成。

[3] Trevisan, L. (2009; preprint 2008).

Max Cut and the Smallest Eigenvalue. STOC 2009.

[arXiv:0806.1978](#)。非負重み Max-Cut とスペクトルの接点。

[4] Rampásek, L. et al. (2022).

Recipe for a General, Powerful, Scalable Graph Transformer. NeurIPS 2022.

[arXiv:2205.12454](#)。GraphGPS。効率的 attention を選べる構成。

[5] Garmendia, A. I. et al. (2024).

MARCO: A Memory-Augmented Reinforcement Framework for Combinatorial Optimization. IJCAI 2024, 6931–6939.

[公開本文](#)。Section 5 の Graph Transformer。

[6] Barrett, T. D.; Clements, W. R.; Foerster, J. N.; Lvovsky, A. I. (2020).

Exploratory Combinatorial Optimization with Reinforcement Learning. AAAI, 34(04), 3243–3250.

[出版版 / 本文・補足資料 / 公式実装](#)。観測・報酬・Table 1・SimCIM 比較。

[7] 本リポジトリの入力・記録・実装。

input/G22.txt, G55.txt, G70.txt, modules/CIM.py、8 月 23 日の CIM 解析、9 月 14 日の GA 世代別多様性測定。本文の性能値は既存結果で、新規実験ではない。

[8] NetworkX : k-core の定義と実装。

[公式ドキュメント](#)。本資料の集計環境は NetworkX 3.6.1。

今回の追加集計と、図の再現

図 5 の内訳は data/decomposition.json に保存した。図 9 の次数、図 11 の G70 閉路、図 13 の説明用固有値も同梱データから再現できる。説明用の小グラフと、実入力の集計値は各図の説明に明記した。

make_second_revision_figures.py で追加集計と図を再生成できる。旧版から継続する図のコード、TeX、データはソース ZIP に同梱。赤字ごとの反映先は redline_response.md に記録した。